The PostgreSQL parameter max_connections[9] can be used to control the maximum number of connections in use. Changing this value requires restarting PostgreSQL.

A default value of 100 reflects a smaller PostgreSQL database server. A modern production PostgreSQL server might have a higher value like 500 or more.

The steps to simulate connection exhaustion are:

- Set the max_connections to a low value of 3
- Open 3 psql connections to the server, and run a long query
- Open a 4th connection, observing that the connection fails to open because all connections are in use

Run this statement in psql:

sql/show_max_connections.sql

```
-- default: 100
SHOW max_connections;
```

Now change it. Edit postgresql.conf, setting max_connections to 3. Edit the value of superuser_reserved_connections to be 1 less than the value of max_connections, which is a requirement from PostgreSQL for this exercise. Restart PostgreSQL:

sh/postgresql_conf_edit_max_connections.sh

```
# use `vim` or another text editor
vim /path/to/postgresql.conf

# set to low value
max_connections = 3
superuser_reserved_connections = 2

# restart PostgreSQL, required when changing `max_connections`
pg_ctl restart -D /path/to/data_directory/
```

Now that you've edited max_connections to be 3, when you attempt to run a query using a 4th connection you'll see an error. To simulate this run the following script from your Terminal by copying and pasting it into a file and then running it with "sh exhaust_database_connection.sh".

sh/exhaust_database_connections.sh

```
#!/bin/bash

# set max_connections to 3

query="SELECT pg_sleep(30)"

for run in {1..4}; do
  echo "running query ${query}. times: ${run}"
  psql \
```

9. https://postgresqlco.nf/doc/en/param/max_connections/